File-System

Mata Kuliah Sistem Operasi

Dosen Pengampu: Triawan Adi Cahyanto, M.Kom

Program Studi Teknik Informatika

Universitas Muhammadiyah Jember

2026

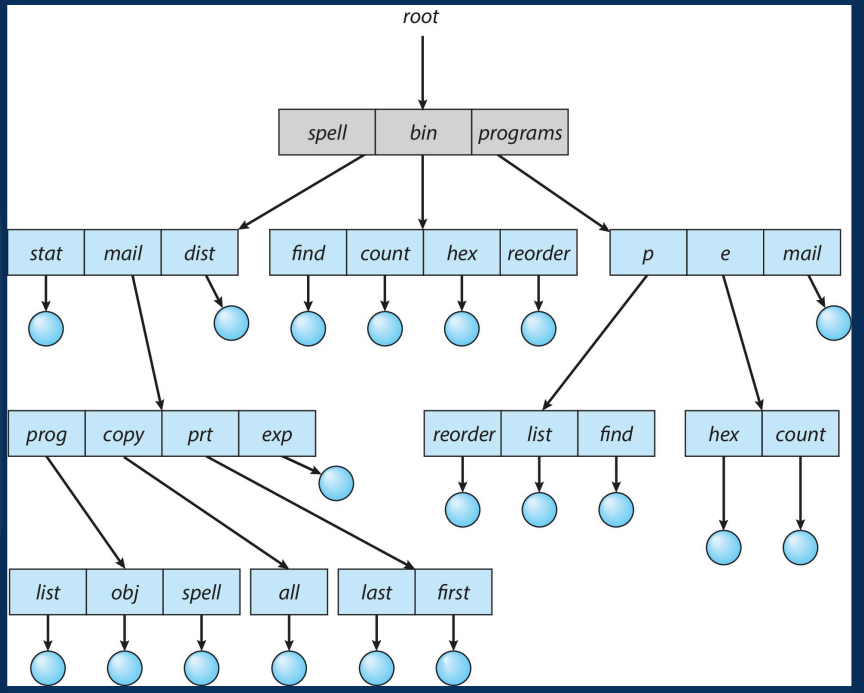
Sistem Operasi

UM Jember

File-System Interface

Antarmuka yang menghubungkan pengguna dengan sistem file, menyediakan cara untuk mengorganisasi, mengakses, dan mengelola data dalam storage.

- Abstraksi untuk pengaksesan data
- Struktur direktori dan file
- Mekanisme proteksi dan akses



Struktur File System

File Concept

Apa itu File?

Koleksi bernama dari informasi yang berhubungan, disimpan di storage media.

- Text
- Binary
- Executable

Atribut File

Name
Nama unik file

Type
Tipe file

Location
Alamat di disk

Size
Ukuran file

Protection
Hak akses file

Time & Date
Waktu dibuat/diubah

User Identification
Informasi pemilik file

File Concept

Tipe File

Text File	.txt, .md, .csv
Binary File	.bin, .dat
Executable File	.exe, .app
Source Code	.c, .py, .java
Database	.db, .sql
Directory	Struktur folder

Operasi File

- Create
- Delete
- Open
- Close
- Read
- Write
- Seek
- Append
- Truncate
- Rename

Setiap operasi file memerlukan **permission** dan **file handle** yang valid.

Access Methods

Sequential Access

Baca dan tulis secara berurutan dari awal ke akhir

Tape | Sequential

Direct Access

Akses langsung ke blok tertentu dalam file

Disk | Random

Indexed Sequential

Kombinasi sequential dan indexed access

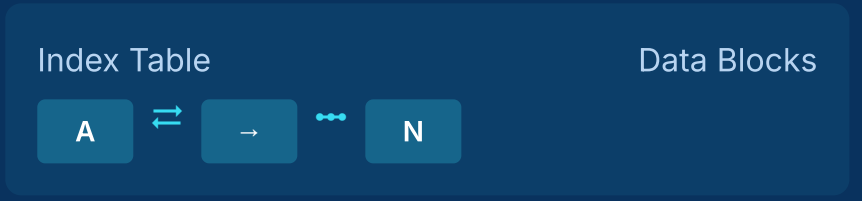
Database | Efficient

Visualisasi Akses

Sequential Access

Direct Access

Indexed Sequential



Directory Structure

Single-Level

Semua file di root, sederhana tapi terbatas

Two-Level

Setiap user punya direktori sendiri, lebih terorganisir

Tree-Structured

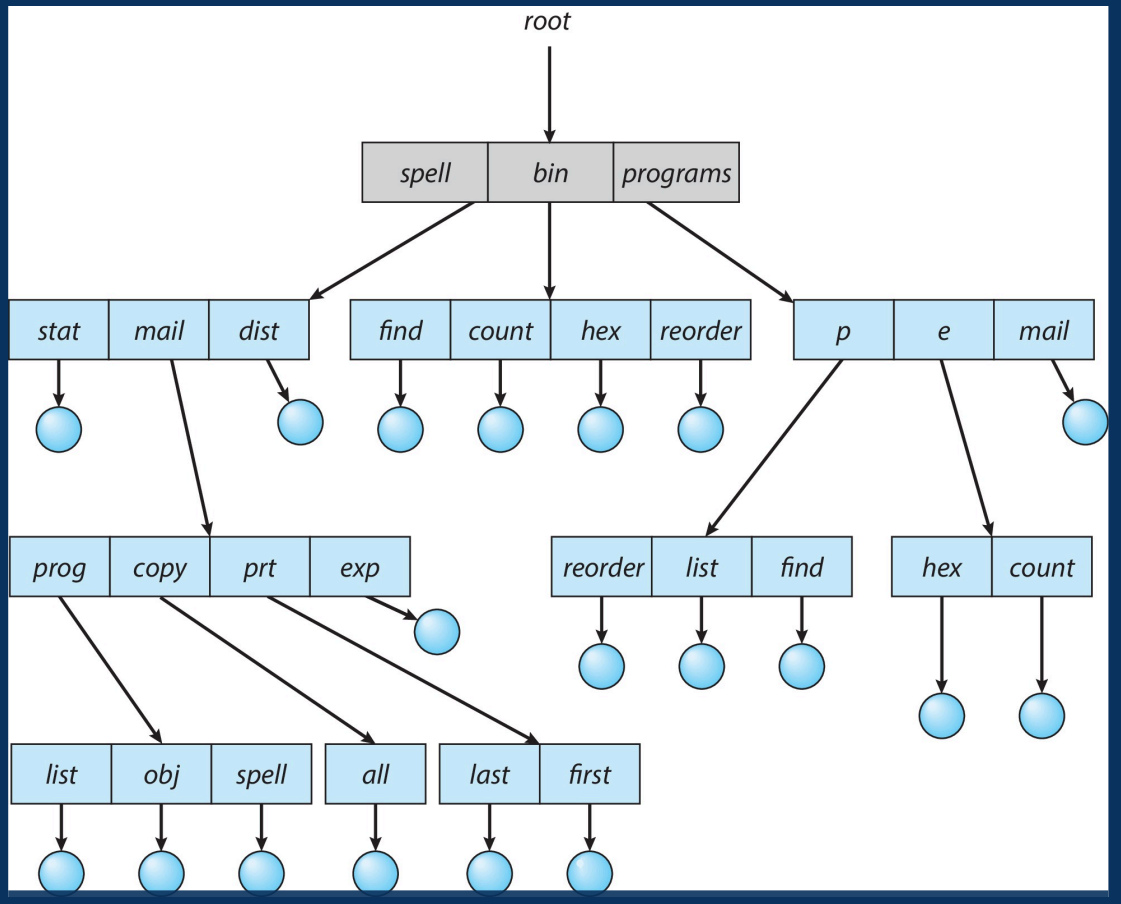
Hirarki banyak level, fleksibel dan umum digunakan

Acyclic-Graph

File bisa punya multiple paths tanpa siklus

General-Graph

Dengan siklus, komplex dan jarang digunakan



Contoh Struktur Direktori Tree

Protection

Mengontrol akses file untuk mencegah akses yang tidak sah

Keamanan Data

Permission

Read | Write | Execute

Ownership

Owner	Group	Others
Pemilik	Grup	Lainnya

Access Control Lists (ACL)

Daftar yang mengatur akses untuk setiap user atau grup secara individual

Password

Proteksi dengan password

Encryption

Enkripsi data file

Memory-Mapped Files

Konsep

File dihubungkan langsung ke alamat virtual memory, memungkinkan akses seperti memory biasa

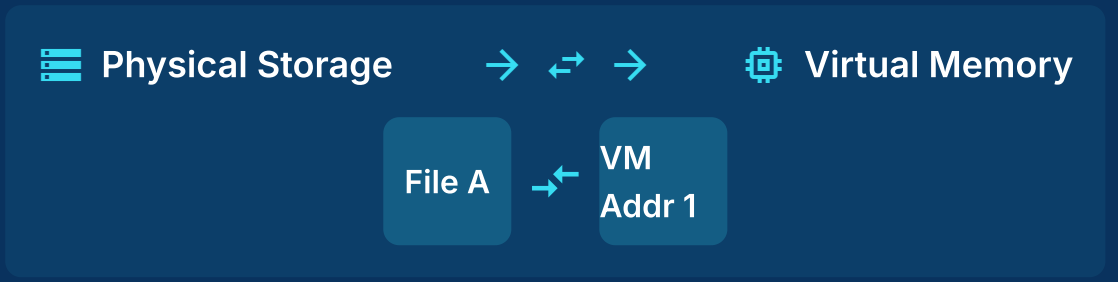
Process Mapping

File dipetakan ke process address space, setiap process bisa mengakses file melalui alamat virtualnya

Virtual Memory & File Share

File bisa di-share antar process melalui memory mapping yang sama

Visualisasi Memory Mapping



Kelebihan

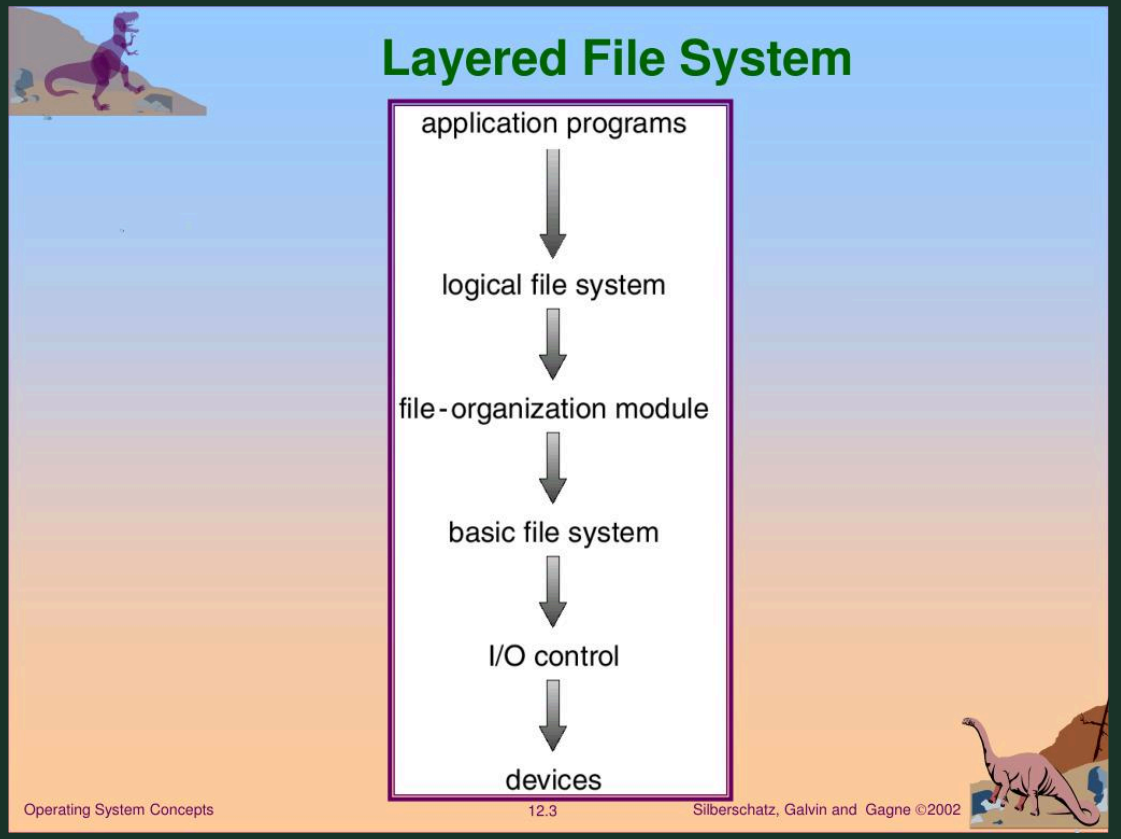
- Efisien dalam I/O
- Akses sangat cepat
- Mudah sharing data
- Inter-process communication

File-System Implementation

Implementasi
Sistem File

- Bagaimana OS mengelola dan menyimpan file di storage
- Struktur dan alokasi ruang di disk
- Operasi, performa, dan mekanisme recovery

- Directory
- Allocation
- Free Space
- Recovery



Arsitektur Layered File System

File-System Structure

Boot Control Block

Boot block yang berisi informasi untuk mem-boot sistem operasi

Partition Control Block

Informasi partisi disk seperti ukuran, tipe, dan lokasi blok

File Control Block

FCB/Inode yang berisi metadata dan informasi lokasi file

1 Boot Block

Pertama kali diakses saat komputer dinyalakan

2 Superblock / Partition Table

Informasi partisi dan metadata file system

3 Directory Structure

Struktur direktori yang mengorganisir file

4 Inode Table

Tabel inode untuk file metadata

5 Data Blocks

Blok-blok data yang berisi isi file

File-System Operations

Create
Membuat file baru, alokasi FCB dan blok data

Delete
Menghapus file dan FCB, membebaskan ruang

Open
Membuka file, load FCB ke memory

Close
Menutup file, simpan ke disk

Read
Membaca data dari file ke memory buffer

Write
Menulis data dari buffer ke file

Seek
Memindahkan pointer posisi file

Alur Operasi File

1. **Create File**
   Alokasi blok dan FCB baru
2. **Open File**
   Load FCB ke memory
3. **Read/Write/Seek**
   Operasi data melalui pointer
4. **Close File**
   Simpan kembali ke disk
5. **Delete File**
   Bebaskan FCB dan blok data

Directory Implementation

Linear List

- Simpel dan mudah diimplementasikan
- Pencarian lambat (O(n))
- Penggunaan space tidak efisien

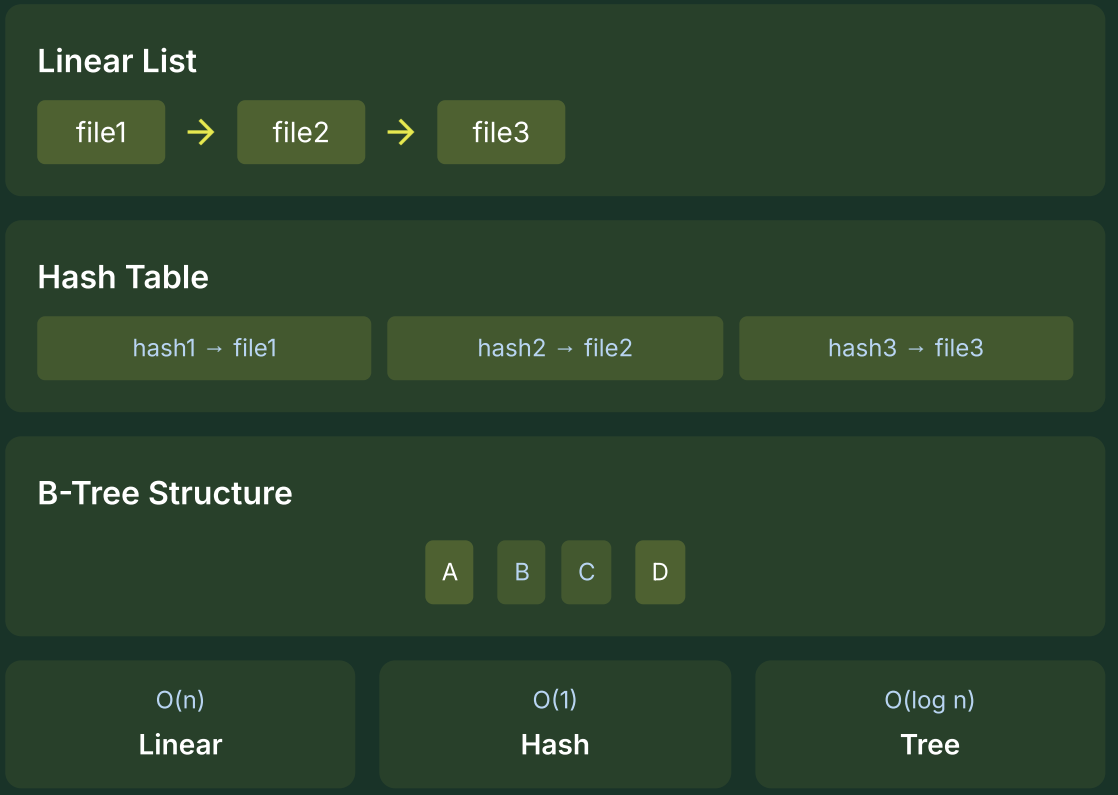
Hash Table

- Pencarian sangat cepat (O(1))
- Efisien untuk direktori besar
- Kompleksitas implementasi tinggi

B-Tree & Balanced Tree

- Pencarian cepat dan efisien
- Support range query
- Kompleksitas tinggi

Visualisasi Struktur



Allocation Methods – Contiguous

Konsep

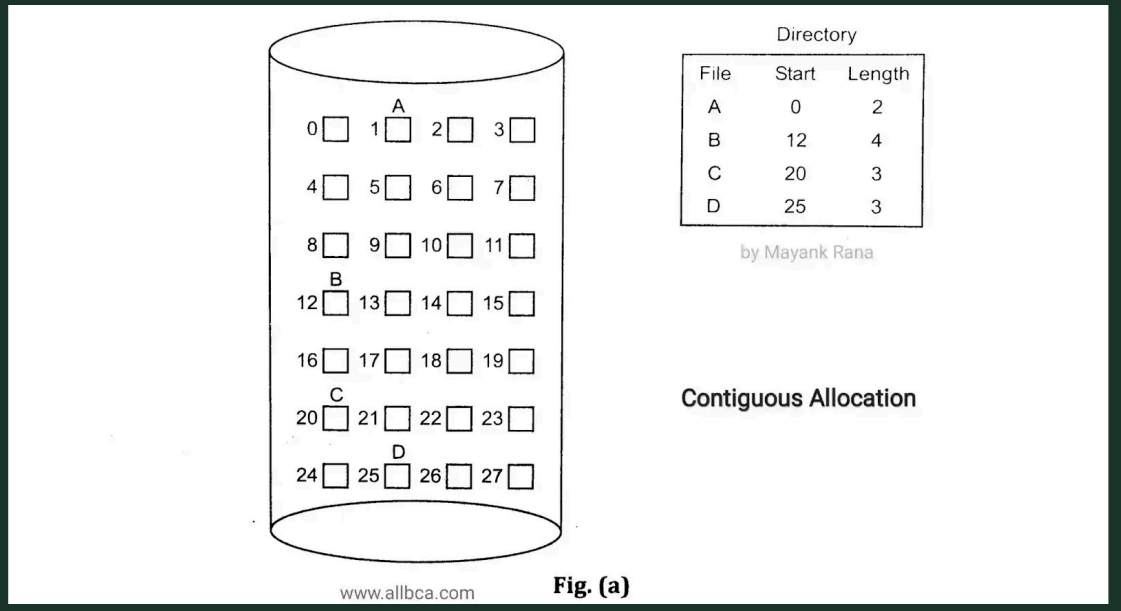
File disimpan dalam blok-blok yang bersebelahan secara berurutan

Kelebihan

- Akses sangat cepat
- Sequential access optimal
- Simpel diimplementasikan
- Minimal overhead

Kekurangan

- External fragmentation
- File tidak bisa grow
- Membutuhkan compaction
- Fixed size file



Struktur FCB

Starting Block → Length

Akses Sequential	Direct Access
O(1)	O(1)

Allocation Methods – Linked

Konsep

Setiap blok file berisi data dan pointer ke blok berikutnya

Kelebihan

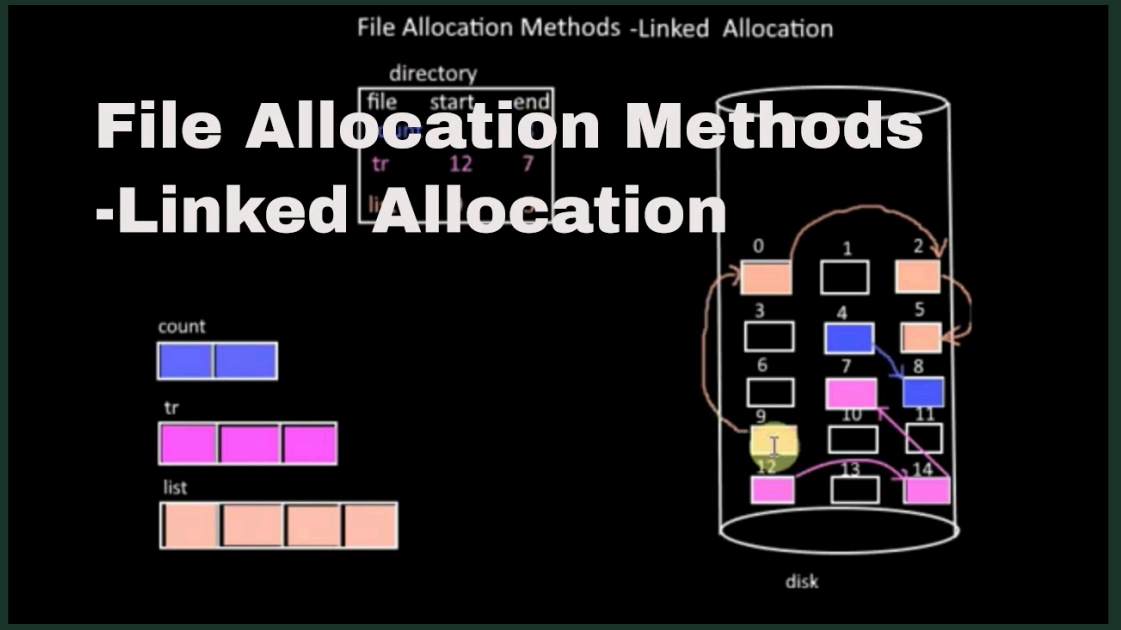
- No external fragmentation
- File bisa grow dinamis
- Efisien untuk file besar

Kekurangan

- Akses random lambat
- Pointer overhead
- Data recovery sulit

Struktur Linked

Start → Block1→Block2 → ...→NULL



Akses Sequential	Direct Access
O(1)	O(n)

Allocation Methods – Indexed

Konsep

File memiliki index block berisi pointer ke semua data blocks

Kelebihan

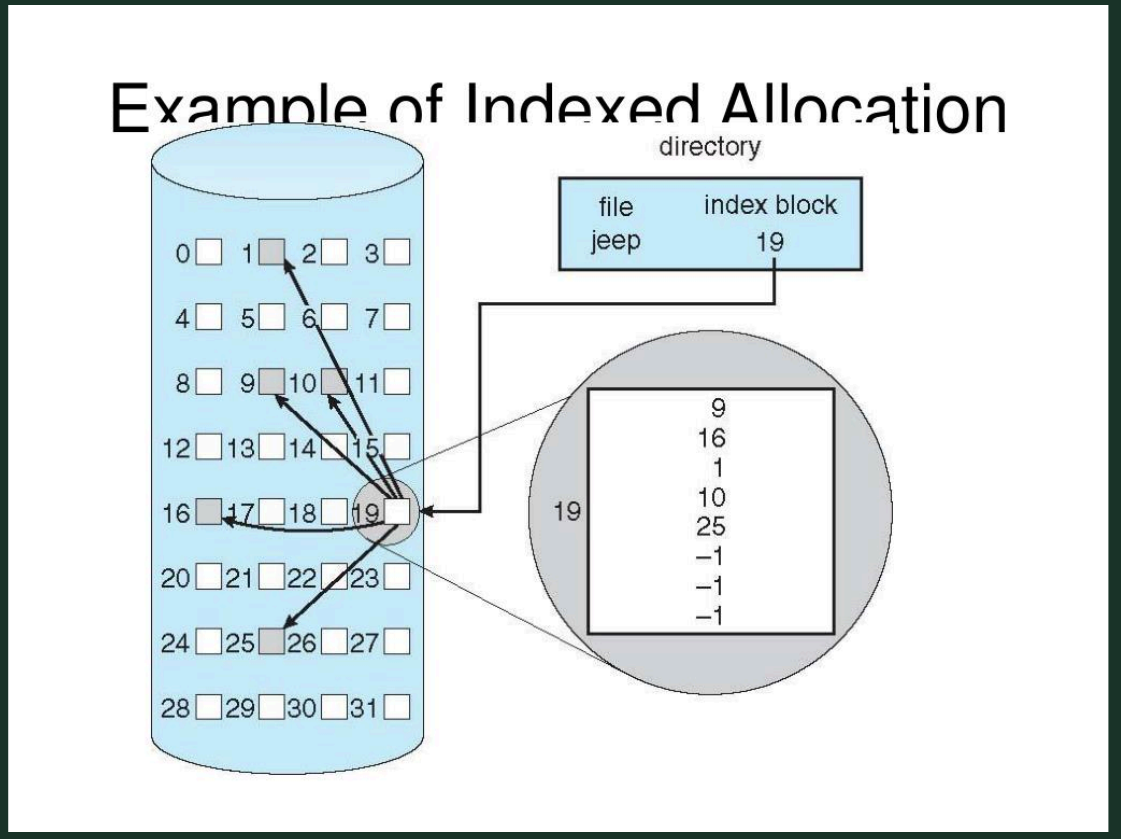
- Akses random cepat
- No external fragmentation
- File bisa grow

Kekurangan

- Index block overhead
- Kompleksitas tinggi
- Wasted space

Struktur Indexed

Index Block → [ptr1,ptr2,...] → Data Blocks



Akses Sequential	Direct Access
O(1)	O(1)

Free-Space Management

Bit Vector

Setiap bit merepresentasikan status blok (0=free, 1=used)

0	1	0	0

- Simple & fast
- Wasted space

Linked List

Blok free dihubungkan dalam linked list

B1 → B3 → B7

- Efficient space
- Random access slow

Grouping

Blok di-group dalam satu unit

[n,addr]

- Space efficient
- Moderate complexity

Counting

Menyimpan count blok free consecutive

[count,start]

- Very efficient
- Slow fragmentation

Comparison

Space Usage	Bit Vector worst
Speed	Bit Vector fastest
Flexibility	Linked List most flexible
Complexity	Bit Vector simplest

Efficiency and Performance

Faktor Performa

- Disk Speed
- Cache Size
- Algorithm

Caching

- Buffer cache
- Data cache

Buffering

- Read-ahead
- Write-back

Read-ahead

Prediksi akses data berikutnya untuk mempercepat operasi

Current → Next → Next

Disk Scheduling

Compression

Striping

Performance Impact

Cache Hit Ratio	90%
Disk Access Time	↓ 70%
I/O Throughput	↑ 80%

Best Practices

- Use cache size > 64MB
- Enable read-ahead
- Optimize disk scheduling

Recovery

Konsistensi File System

Metadata dan data harus tetap konsisten setelah system failure

Journaling

Log transaction sebelum eksekusi

Write Log → Execute → Commit

Checkpointing

Simpan state berkala untuk recovery

CP1 | CP2 | CP3

Backup & Restore

Restore dari backup file terakhir

Repair Tools

- fsck (Linux)
- chkdsk (Windows)

Why Important?

Prevent data loss | System reliability | Fast recovery

Recovery Process

1. Deteksi failure
2. Analisa journal
3. Redo committed
4. Undo uncommitted
5. Verify consistency
6. System ready

Example: The WAFL File System

Write Anywhere File Layout

File system yang dirancang untuk NAS dengan fitur snapshot dan write-anywhere allocation

Snapshot

Copy read-only of file system state

T1 | T2 | T3

Write-Anywhere

Blok baru untuk setiap modifikasi

Old → New

Copy-on-Write

Hanya blok yang dimodifikasi yang ditulis ulang, blok yang tidak dimodifikasi tetap di-share antar snapshot

- Efisien
- Cepat
- Snapshot Support

WAFL Architecture

Inode File

Berisi metadata file

Block Map

Mapping blok fisik

Snapshot Info

Informasi snapshot

RAID Array

Redundant storage

Keunggulan

- Tidak ada fragmentasi
- Performa tinggi
- Restore cepat
- Space efficient

Summary File-System Implementation

Structure

- Boot block
- FCB / Inode
- Data blocks
- Superblock

Allocation

- Contiguous
- Linked
- Indexed
- Performance trade-off

Free Space

- Bit vector
- Linked list
- Grouping
- Counting

Operations

- Create / Delete
- Open / Close
- Read / Write
- Seek / Append

Directory

- Linear list
- Hash table
- B-Tree
- Search efficiency

Recovery

- Journaling
- Checkpointing
- Backup & restore
- Consistency check

Key Takeaway: File system implementation membutuhkan keseimbangan antara efisiensi, performa, dan reliability

File-System Internals

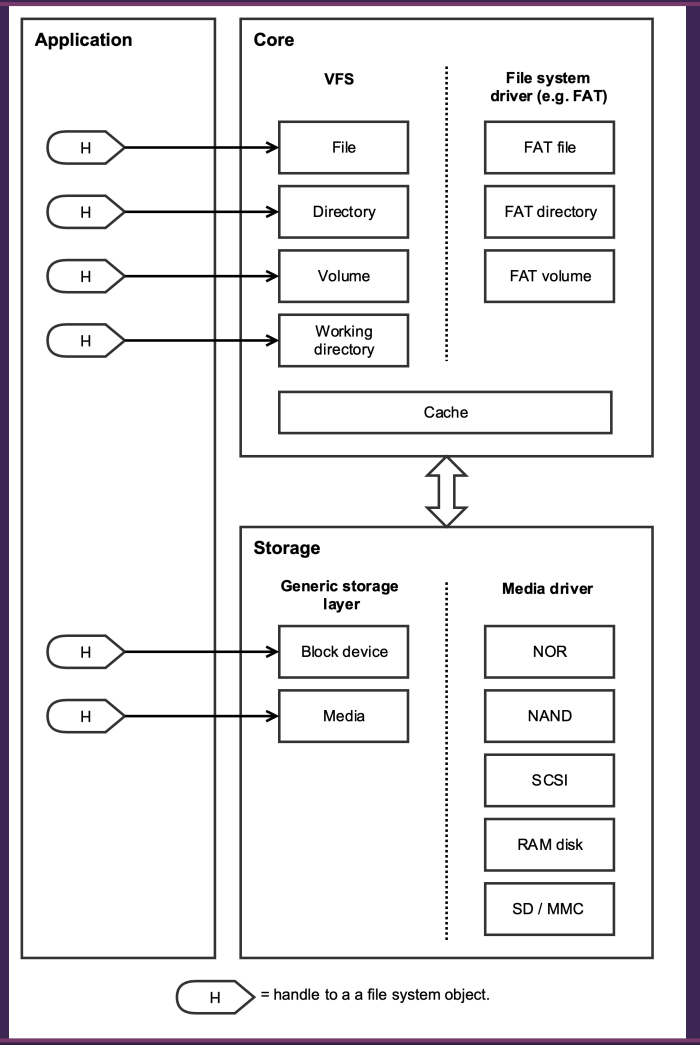
Overview

Internal architecture dan implementasi detail file system yang menghubungkan aplikasi dengan storage media

Key Components

- File Systems
- Partitions
- VFS Layer
- Consistency
- Mounting
- File Sharing
- Remote Systems
- NFS Protocol

Layered Architecture memisahkan concerns dan menyediakan abstraction untuk kemudahan management



File Systems

FAT

File Allocation Table

Karateristik

- ✓ Simple structure
- ✓ High compatibility
- ✓ Flash drives
- ✓ No permissions

NTFS

New Technology File System

Karateristik

- ✓ Security features
- ✓ Compression
- ✓ Large files
- ✓ Windows default

ext4

Fourth Extended File System

Karateristik

- ✓ Journaling
- ✓ Large volumes
- ✓ Linux default
- ✓ Ext4 features

HFS+

Hierarchical File System Plus

Karateristik

- ✓ MacOS support
- ✓ Compression
- ✓ Large files
- ✓ Legacy support

APFS

Apple File System

Karateristik

- ✓ Modern design
- ✓ Snapshots
- ✓ Encryption
- ✓ MacOS default

ZFS

Zettabyte File System

Karateristik

- ✓ Enterprise
- ✓ Data integrity
- ✓ Snapshots
- ✓ Sun/Oracle

File-System Mounting

Konsep

Menghubungkan file system ke direktori dalam hierarki file yang sudah ada

Mount Point

Direktori tempat FS di-mount

Root FS

File system utama

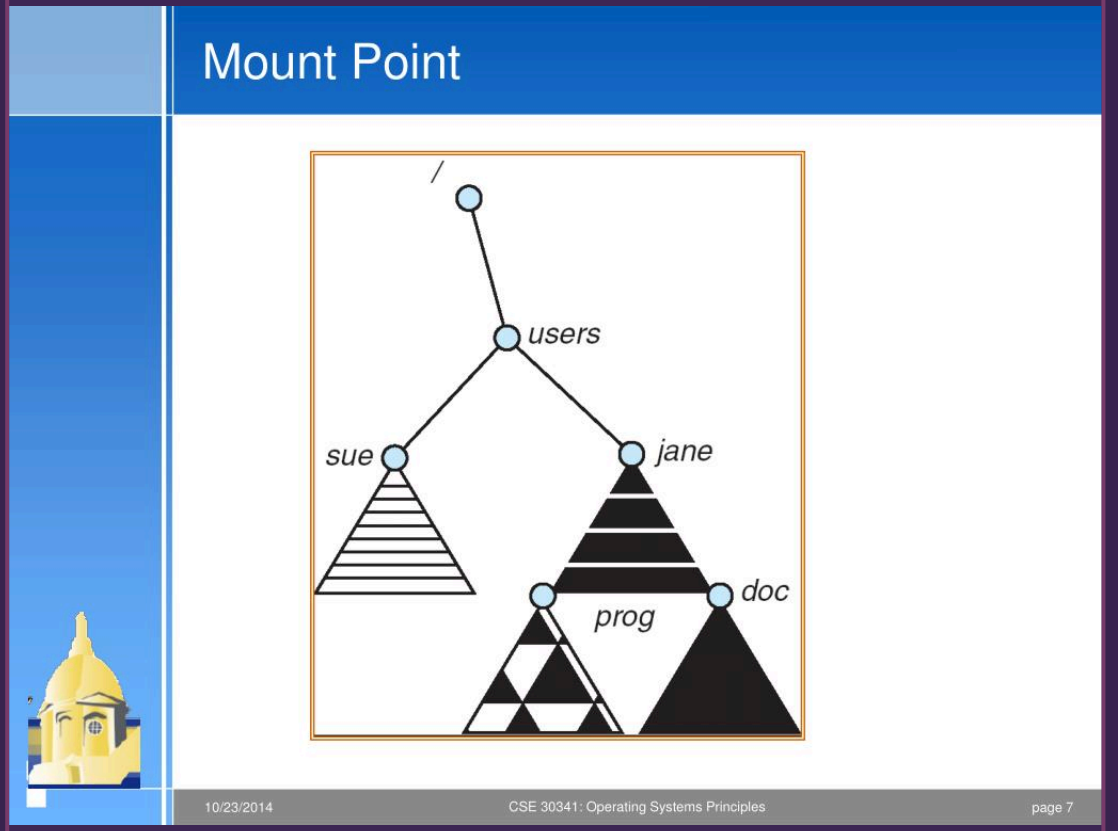
Device

Storage yang di-mount

Proses Mounting

1. Buat mount point (direktori kosong)
2. Mount file system ke mount point
3. Akses file melalui path mount point

Hasil: User dapat mengakses file system melalui path yang familiar tanpa mengetahui device fisiknya



Mount Point

Partitions and Mounting

Partisi Disk

Membagi fisik disk menjadi bagian-bagian logis yang terpisah

Logical Volumes

Abstraksi di atas partisi fisik

Swap Space

Virtual memory extension

Mounting Process

- Kernel mount tables menyimpan info
- VFS layer handle mounting
- Multiple FS pada single disk

Keunggulan: Isolasi data, boot independence, dan fleksibilitas management

Contoh Struktur Disk

/dev/sda1
Boot Partition

/dev/sda2
Root Filesystem

/dev/sda3
Swap Partition

/dev/sda4
Data Partition

Physical Disk

File Sharing

Konsep

Multiple users dapat mengakses dan berbagi file yang sama secara simultan

File Locking

Kontrol konkurensi

Distributed

Akses jaringan

Semantik Konsistensi

Unix
Semua perubahan visible

Session
Perubahan per session

Immutable
Read-only sharing

Keuntungan: Kolaborasi, efisiensi storage, dan pengurangan redundancy

File Locking Mechanism

Pessimistic
Lock sebelum akses

Optimistic
Konflik diatasi kemudian

Distributed File Systems

- NFS (Network File System)
- AFS (Andrew File System)
- SMB/CIFS
- DFS (Distributed)

Virtual File Systems

VFS Layer

Abstraction layer yang menyediakan interface umum untuk berbagai file system

Abstraction

Hide implementation details

VFS Cache

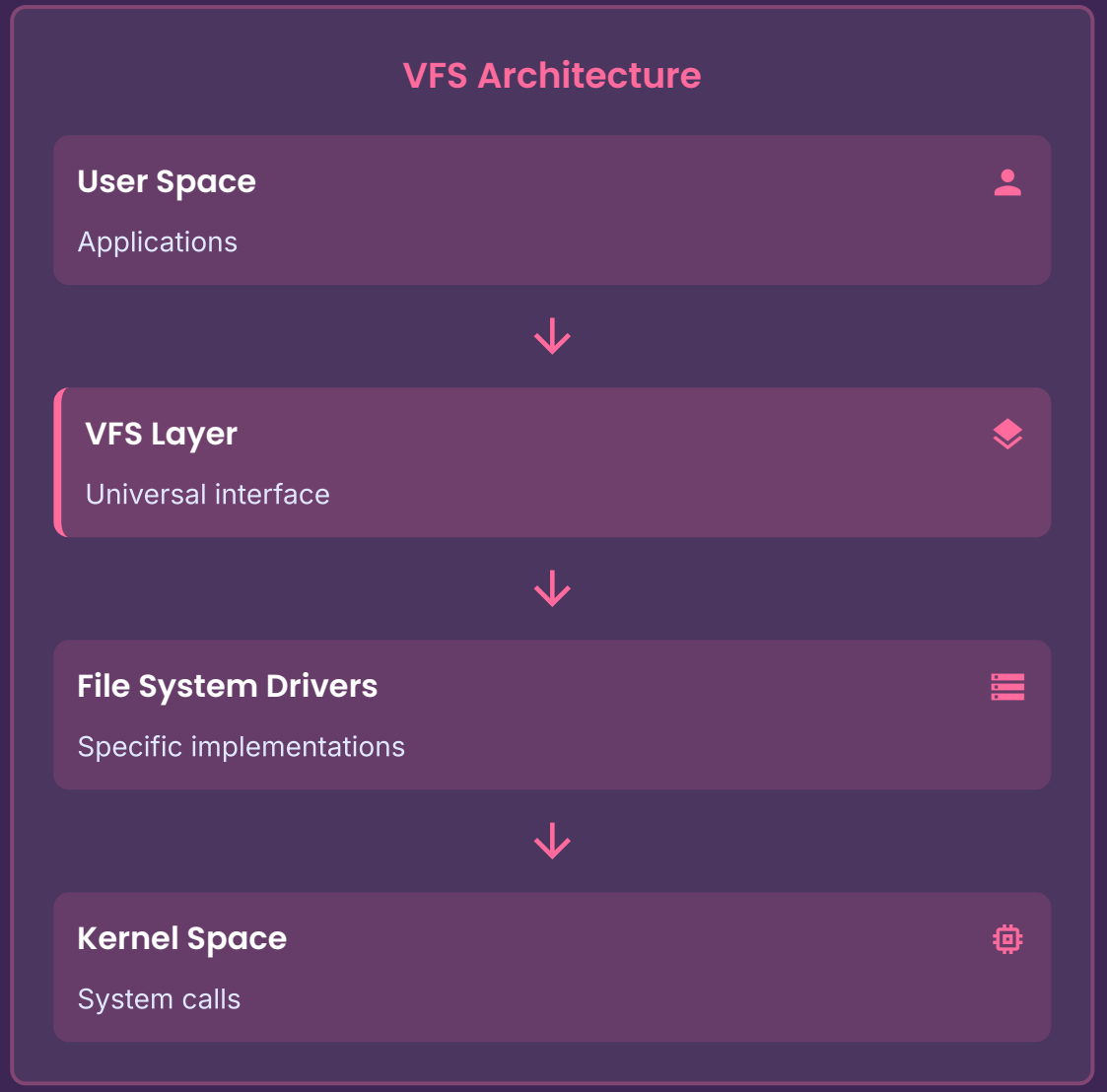
Dentry & inode cache

Multiple File Systems

ext4	NTFS	FAT
NFS	HFS+	...

Keunggulan: Modularity, transparency, dan kemudahan support

VFS Architecture



Remote File Systems

Konsep

Akses file pada mesin lain melalui jaringan sebagai jika file lokal

Client-Server

Architecture model

Distributed

Multiple servers

Protocols

NFS
Network File System

SMB/CIFS
Windows sharing

Manfaat: Kolaborasi, backup terpusat, dan resource sharing

Challenges

- **Network Latency**
  Delay jaringan mempengaruhi performa
- **Consistency**
  Menjaga konsistensi data terdistriusi
- **Security**
  Authentication dan encryption
- **Failure Handling**
  Network dan server failures

Consistency Semantics

Konsep

Kapan perubahan file menjadi visible ke process lain

Mengapa Penting?

- Konsistensi data antar process
- Prediktability akses file
- Behavior sharing file

Fokus: Trade-off antara konsistensi dan performa

Unix Semantics

REAL-TIME

Visibility
Segera ke semua process

Pros
Konsistensi tinggi

Session Semantics

ON CLOSE

Visibility
Setelah file close

Pros
Performa lebih baik

Immutable-Shared-Files

READ-ONLY

Visibility
File tidak dapat diubah

Pros
Sederhana dan aman

NFS – Network File System

Protocol

File sharing berbasis RPC (Remote Procedure Call)

Client	Server	VFS Layer

Key Operations

mount	lookup	read
write	getattr	setattr

Keunggulan: Cross-platform, transparansi, caching

NFS Architecture

Summary - File-System Internals

File Systems
FAT, NTFS, ext4, HFS+, APFS, ZFS - berbagai tipe file system dengan karakteristik berbeda

Mounting & Partitions
Process mounting, mount points, partisi disk, logical volumes, dan swap space

File Sharing
File locking, concurrency control, dan consistency semantics (Unix, Session, Immutable)

VFS Layer
Abstraction layer, VFS cache, dan kemudahan support multiple file systems

Remote File Systems
NFS, SMB/CIFS, client-server architecture, dan distributed file systems

Key Takeaways

- File system adalah komponen kunci dalam OS
- Abstraction memungkinkan fleksibilitas
- VFS layer mempermudah implementasi
- Remote file systems modern computing

Terima Kasih

Mata Kuliah Sistem Operasi

Dosen: Triawan Adi Cahyanto, M.Kom
Teknik Informatika - Universitas Muhammadiyah Jember

2026